

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



Sistema de Autorización de Documentos Tributarios Electrónicos

PONDERACIÓN: 100

Horas Aproximadas: 40

1. Resumen Ejecutivo

Este proyecto consiste en el desarrollo de un sistema para la Superintendencia de Administración Tributaria que permite la autorización de Documentos Tributarios Electrónicos (DTE). El sistema está compuesto por un frontend web desarrollado con Django y un backend API implementado con Flask. El frontend permite cargar archivos XML con solicitudes de autorización, visualizar estadísticas y generar reportes, mientras que el backend procesa las solicitudes, valida la información, asigna códigos de autorización y almacena los resultados en archivos XML. La solución implementa programación orientada a objetos en Python, manejo de expresiones regulares y generación de gráficas interactivas.

2. Competencia que desarrollaremos

- Desarrollo de APIs utilizando el protocolo HTTP
- Implementación de aplicaciones web con Django (MVT)
- Desarrollo de servicios web con Flask
- Manejo de archivos XML para intercambio de información
- Uso de expresiones regulares para procesamiento de texto
- Generación de reportes y visualizaciones gráficas
- Implementación de arquitecturas cliente-servidor

3. Objetivos del Aprendizaje

3.1 Objetivo General

Desarrollar una solución integral que implemente un API que brinde servicios utilizando el Protocolo HTTP bajo el concepto de programación orientada a objetos (POO) y el uso de bases de datos.

3.2 Objetivos Específicos

- Implementar un API a través de lenguaje Python que pueda ser consumida utilizando el protocolo HTTP
- Utilizar el paradigma de programación orientada a objetos para construir software
- Utilizar bases de datos para almacenar información de forma persistente
- Utilizar archivos XML como insumos para la comunicación con el API desarrollado
- Utilizar expresiones regulares para extraer contenido de texto

4. Enunciado del Proyecto

4.1 Descripción del problema a resolver

La Superintendencia de Administración Tributaria requiere un software que pueda ser consumido desde Internet como un servicio para la autorización de Documentos Tributarios Electrónicos (DTE). El sistema debe recibir solicitudes de autorización en formato XML, validar la información (NIT emisor, NIT receptor, cálculos de IVA y total), asignar un número único de autorización con formato `yyyymmdd####` y almacenar los resultados en archivos

XML.

El sistema debe ser capaz de identificar y contabilizar errores en las solicitudes (NIT inválido, cálculos incorrectos, referencias duplicadas) y generar estadísticas sobre las autorizaciones procesadas. Además, debe proporcionar funcionalidades para consultar datos, generar resúmenes por fecha y NIT, y crear reportes gráficos.

4.2 Alcance del proyecto

El proyecto se divide en dos componentes principales:

1. Frontend (Programa 1):

- Interfaz web desarrollada con Django (MVT)
- Carga de archivos XML con solicitudes de autorización
- Consulta de datos almacenados en el archivo autorizaciones.xml
- Generación de resúmenes de IVA por fecha y NIT
- Generación de resúmenes por rango de fechas
- Visualización gráfica de datos utilizando chartjs
- Generación de reportes en formato PDF

2. Backend (Servicio 2):

- API desarrollada con Flask
- Procesamiento de solicitudes de autorización
- Validación de información (NIT, cálculos de IVA y total)
- Asignación de números de autorización
- Almacenamiento de resultados en archivos XML
- Generación de estadísticas y reportes

4.3 Requerimientos técnicos

- Uso obligatorio de programación orientada a objetos (POO) en Python
- Frontend desarrollado con Django (MVT)
- Backend desarrollado con Flask
- Uso de expresiones regulares para procesamiento de texto
- Manejo de archivos XML para intercambio de información
- Generación de gráficas utilizando chartjs
- Generación de reportes en formato PDF
- Uso de versionamiento (GitHub) con 4 releases durante el desarrollo

4.4 Entregables

Tipo	Descripción
Frontend (Django)	Aplicación web con interfaz para carga de archivos, consulta de datos, generación de resúmenes y visualización gráfica
Backend (Flask)	API para procesamiento de solicitudes de autorización, validación de información y almacenamiento de resultados
Archivo de salida	Archivo XML con estructura definida para almacenar resultados de autorizaciones

Informe Técnico	Ensayo de 4-7 páginas explicando la solución implementada
Documentación Técnica	Diagrama de clases, diagramas de actividades y diagrama de despliegue
Código Fuente	Repositorio en GitHub con 4 releases mostrando el avance del proyecto

5. Metodología

El desarrollo del proyecto seguirá las siguientes etapas:

1. **Análisis (Semana 1)**: Comprensión del problema, identificación de requerimientos y diseño inicial de la solución.
2. **Diseño (Semana 1-2)**: Creación del diagrama de clases, diagramas de actividades y diagrama de despliegue.
3. **Implementación del Backend (Semana 2)**: Desarrollo de la API con Flask para procesamiento de solicitudes.
4. **Implementación del Frontend (Semana 3)**: Desarrollo de la interfaz web con Django.
5. **Integración y Pruebas (Semana 3-4)**: Conexión entre frontend y backend, pruebas de funcionalidad.
6. **Generación de Reportes y Gráficas (Semana 4)**: Implementación de visualizaciones y reportes PDF.
7. **Documentación (Semana 4)**: Elaboración del ensayo, diagramas y preparación de la entrega final.

6. Desarrollo de Habilidades Blandas

6.2 Proyectos Individuales

6.2.1 Autogestión del Tiempo

Este proyecto requiere una planificación efectiva para cumplir con los 4 releases requeridos. El estudiante deberá organizar su tiempo para avanzar constantemente, dedicando aproximadamente 10 horas semanales durante el mes de desarrollo.

6.2.2 Responsabilidad y Compromiso

El estudiante asumirá la responsabilidad total del desarrollo, desde el análisis inicial hasta la entrega final, demostrando compromiso con la calidad del código y la documentación.

6.2.3 Resolución de Problemas

El proyecto presenta desafíos técnicos como la implementación de una arquitectura cliente-servidor, validación de datos y generación de reportes gráficos, que requerirán investigación y soluciones creativas.

6.2.4 Reflexión Personal

Al finalizar, el estudiante evaluará su proceso de desarrollo, identificando fortalezas y áreas de mejora para futuros proyectos.

7. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto	4 de abril	4 de abril
Release 1: Análisis y diseño	4 de abril	11 de abril
Release 2: Implementación del Backend	12 de abril	18 de abril
Release 3: Implementación del Frontend	19 de abril	25 de abril
Release 4: Integración, Pruebas y Documentación	26 de abril	1 de mayo
Entrega Final	2 de mayo	2 de mayo

8. Rúbrica de Calificación

8.1 Requisitos para optar a la calificación

Tema	Descripción	Cumple (Si/No)
Cumplimiento de la tecnología establecida	Desarrollo en Python utilizando POO	
	Frontend desarrollado con Django	
	Backend desarrollado con Flask	
Gestión y entregas del proyecto	4 releases en GitHub según cronograma	
	Repositorio con nombre IPC2_Proyecto3_#Carnet	
Documentación obligatoria	Ensayo de 4-7 páginas	
	Diagrama de clases, diagramas de actividades y diagrama de despliegue	
Pruebas y funcionalidad mínima	Sistema funcional que cumpla con los requisitos establecidos	

8.2 Resumen de Puntuaciones

Área	Puntos Totales	Puntos Obtenidos
Backend (Flask)	35	
Postman	25	
Frontend	35	
Documentación	5	
TOTAL	100	

8.3 Detalle de la Calificación

Criterio	Descripción	Puntos Máximos	Puntuación Obtenida
Backend (Flask)		35	
Lectura correcta del archivo de entrada	Ignorar DTE si el formato del XML es incorrecto	5	
Expresión Regular para obtención de fecha		5	
Validación de errores		10	
Archivo de salida		15	
Postman		25	
Consultar Datos		5	
Resumen de IVA por fecha y NIT		5	
Resumen por rango de fechas		5	
Gráfica		5	
Reporte en PDF		5	
Frontend		35	
Botones (ayuda, enviar, reset)		5	
Creativo e intuitivo		5	
Resúmenes		10	
Gráficas		15	
Documentación		5	
Ensayo		2	
Diagrama de Clases		1	
Diagrama de Actividades		1	
Diagrama de Despliegue		1	
Total		100	

8.4 Valores

En el desarrollo del proyecto, se espera que cada estudiante demuestre honestidad académica y profesionalismo. Por lo tanto, se establecen los siguientes principios:

- Originalidad del Trabajo

Cada estudiante debe desarrollar su propio código y documentación, aplicando los conocimientos adquiridos en el curso.

- Prohibición de Copias y Plagio

Si se detecta la copia total o parcial del código, documentación o cualquier otro entregable, la calificación será de 0 puntos.

Esto incluye la reproducción de código entre compañeros, la reutilización de proyectos de semestres anteriores o el uso de código externo sin la debida referencia.

- Uso Responsable de Recursos Externos

El uso de bibliotecas, frameworks y ejemplos de código externos está permitido, siempre y cuando se referencien correctamente y se comprendan plenamente.

- Revisión y Detección de Plagio

Se podrán utilizar herramientas automatizadas y revisiones manuales para identificar similitudes en los proyectos.

En caso de sospecha, el estudiante deberá justificar su código y demostrar su desarrollo individual.

- Penalizaciones

- Falta de Documentación: -100%
- Uso de Javascript para la comunicación entre backend/frontend: -20pts
- No utilizó Flask (Backend): -100%
- No utilizó Django (Frontend): -100%
- Falta de release: -10% por cada release faltante
- No utilizó POO: -100%

1. Resumen Ejecutivo

Este proyecto consiste en el desarrollo de un sistema para la Superintendencia de Administración Tributaria que permite la autorización de Documentos Tributarios Electrónicos (DTE). El sistema está compuesto por un frontend web desarrollado con Django y un backend API implementado con Flask. El frontend permite cargar archivos XML con solicitudes de autorización, visualizar estadísticas y generar reportes, mientras que el backend procesa las solicitudes, valida la información, asigna códigos de autorización y almacena los resultados en archivos XML. La solución implementa programación orientada a objetos en Python, manejo de expresiones regulares y generación de gráficas interactivas.

2. Competencia que desarrollaremos

- Desarrollo de APIs utilizando el protocolo HTTP
- Implementación de aplicaciones web con Django (MVT)
- Desarrollo de servicios web con Flask
- Manejo de archivos XML para intercambio de información
- Uso de expresiones regulares para procesamiento de texto
- Generación de reportes y visualizaciones gráficas
- Implementación de arquitecturas cliente-servidor

3. Objetivos del Aprendizaje

3.1 Objetivo General

Desarrollar una solución integral que implemente un API que brinde servicios utilizando el Protocolo HTTP bajo el concepto de programación orientada a objetos (POO) y el uso de bases de datos.

3.2 Objetivos Específicos

- Implementar un API a través de lenguaje Python que pueda ser consumida utilizando el protocolo HTTP
- Utilizar el paradigma de programación orientada a objetos para construir software
- Utilizar bases de datos para almacenar información de forma persistente
- Utilizar archivos XML como insumos para la comunicación con el API desarrollado
- Utilizar expresiones regulares para extraer contenido de texto

4. Enunciado del Proyecto

4.1 Descripción del problema a resolver

La Superintendencia de Administración Tributaria requiere un software que pueda ser consumido desde Internet como un servicio para la autorización de Documentos Tributarios Electrónicos (DTE). El sistema debe recibir solicitudes de autorización en formato XML, validar la información (NIT emisor, NIT receptor, cálculos de IVA y total), asignar un número único de autorización con formato `yyyymmdd####` y almacenar los resultados en archivos XML.

El sistema debe ser capaz de identificar y contabilizar errores en las solicitudes (NIT inválido, cálculos incorrectos, referencias duplicadas) y generar estadísticas sobre las autorizaciones procesadas. Además, debe proporcionar funcionalidades para consultar datos, generar resúmenes por fecha y NIT, y crear reportes gráficos.

4.2 Alcance del proyecto

El proyecto se divide en dos componentes principales:

1. Frontend (Programa 1):

- Interfaz web desarrollada con Django (MVT)
- Carga de archivos XML con solicitudes de autorización
- Consulta de datos almacenados en el archivo `autorizaciones.xml`
- Generación de resúmenes de IVA por fecha y NIT
- Generación de resúmenes por rango de fechas
- Visualización gráfica de datos utilizando `chartjs`
- Generación de reportes en formato PDF

2. Backend (Servicio 2):

- API desarrollada con Flask
- Procesamiento de solicitudes de autorización
- Validación de información (NIT, cálculos de IVA y total)
- Asignación de números de autorización
- Almacenamiento de resultados en archivos XML
- Generación de estadísticas y reportes

4.3 Requerimientos técnicos

- Uso obligatorio de programación orientada a objetos (POO) en Python
- Frontend desarrollado con Django (MVT)
- Backend desarrollado con Flask
- Uso de expresiones regulares para procesamiento de texto
- Manejo de archivos XML para intercambio de información
- Generación de gráficas utilizando chartjs
- Generación de reportes en formato PDF
- Uso de versionamiento (GitHub) con 4 releases durante el desarrollo

4.4 Entregables

Tipo	Descripción
Frontend (Django)	Aplicación web con interfaz para carga de archivos, consulta de datos, generación de resúmenes y visualización gráfica
Backend (Flask)	API para procesamiento de solicitudes de autorización, validación de información y almacenamiento de resultados
Archivo de salida	Archivo XML con estructura definida para almacenar resultados de autorizaciones
Informe Técnico	Ensayo de 4-7 páginas explicando la solución implementada
Documentación Técnica	Diagrama de clases, diagramas de actividades y diagrama de despliegue
Código Fuente	Repositorio en GitHub con 4 releases mostrando el avance del proyecto

5. Metodología

El desarrollo del proyecto seguirá las siguientes etapas:

1. **Análisis (Semana 1)**: Comprensión del problema, identificación de requerimientos y diseño inicial de la solución.
2. **Diseño (Semana 1-2)**: Creación del diagrama de clases, diagramas de actividades y diagrama de despliegue.
3. **Implementación del Backend (Semana 2)**: Desarrollo de la API con Flask para procesamiento de solicitudes.
4. **Implementación del Frontend (Semana 3)**: Desarrollo de la interfaz web con Django.
5. **Integración y Pruebas (Semana 3-4)**: Conexión entre frontend y backend, pruebas de funcionalidad.
6. **Generación de Reportes y Gráficas (Semana 4)**: Implementación de visualizaciones y reportes PDF.
7. **Documentación (Semana 4)**: Elaboración del ensayo, diagramas y preparación de la entrega final.

6. Desarrollo de Habilidades Blandas

6.2 Proyectos Individuales

6.2.1 Autogestión del Tiempo

Este proyecto requiere una planificación efectiva para cumplir con los 4 releases requeridos. El estudiante deberá organizar su tiempo para avanzar constantemente, dedicando aproximadamente 10 horas semanales durante el mes de desarrollo.

6.2.2 Responsabilidad y Compromiso

El estudiante asumirá la responsabilidad total del desarrollo, desde el análisis inicial hasta la entrega final, demostrando compromiso con la calidad del código y la documentación.

6.2.3 Resolución de Problemas

El proyecto presenta desafíos técnicos como la implementación de una arquitectura cliente-servidor, validación de datos y generación de reportes gráficos, que requerirán investigación y soluciones creativas.

6.2.4 Reflexión Personal

Al finalizar, el estudiante evaluará su proceso de desarrollo, identificando fortalezas y áreas de mejora para futuros proyectos.

7. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto	4 de abril	4 de abril
Release 1: Análisis y diseño	4 de abril	11 de abril
Release 2: Implementación del Backend	12 de abril	18 de abril
Release 3: Implementación del Frontend	19 de abril	25 de abril
Release 4: Integración, Pruebas y Documentación	26 de abril	1 de mayo
Entrega Final	2 de mayo	2 de mayo

8. Rúbrica de Calificación

8.1 Requisitos para optar a la calificación

Tema	Descripción	Cumple (Si/No)
Cumplimiento de la tecnología establecida	Desarrollo en Python utilizando POO	
	Frontend desarrollado con Django	

	Backend desarrollado con Flask	
Gestión y entregas del proyecto	4 releases en GitHub según cronograma	
	Repositorio con nombre IPC2_Proyecto3_#Carnet	
Documentación obligatoria	Ensayo de 4-7 páginas	
	Diagrama de clases, diagramas de actividades y diagrama de despliegue	
Pruebas y funcionalidad mínima	Sistema funcional que cumpla con los requisitos establecidos	

8.2 Resumen de Puntuaciones

Área	Puntos Totales	Puntos Obtenidos
Backend (Flask)	35	
Postman	25	
Frontend	35	
Documentación	5	
TOTAL	100	

8.3 Detalle de la Calificación

Criterio	Descripción	Puntos Máximos	Puntuación Obtenida
Backend (Flask)		35	
Lectura correcta del archivo de entrada	Ignorar DTE si el formato del XML es incorrecto	5	
Expresión Regular para obtención de fecha		5	
Validación de errores		10	
Archivo de salida		15	
Postman		25	
Consultar Datos		5	
Resumen de IVA por fecha y NIT		5	
Resumen por rango de fechas		5	
Gráfica		5	
Reporte en PDF		5	
Frontend		35	
Botones (ayuda, enviar, reset)		5	
Creativo e intuitivo		5	
Resúmenes		10	
Gráficas		15	
Documentación		5	
Ensayo		2	
Diagrama de Clases		1	

Diagrama de Actividades		1	
Diagrama de Despliegue		1	
Total		100	

8.4 Valores

En el desarrollo del proyecto, se espera que cada estudiante demuestre honestidad académica y profesionalismo. Por lo tanto, se establecen los siguientes principios:

- Originalidad del Trabajo

Cada estudiante debe desarrollar su propio código y documentación, aplicando los conocimientos adquiridos en el curso.

- Prohibición de Copias y Plagio

Si se detecta la copia total o parcial del código, documentación o cualquier otro entregable, la calificación será de 0 puntos.

Esto incluye la reproducción de código entre compañeros, la reutilización de proyectos de semestres anteriores o el uso de código externo sin la debida referencia.

- Uso Responsable de Recursos Externos

El uso de bibliotecas, frameworks y ejemplos de código externos está permitido, siempre y cuando se referencien correctamente y se comprendan plenamente.

- Revisión y Detección de Plagio

Se podrán utilizar herramientas automatizadas y revisiones manuales para identificar similitudes en los proyectos.

En caso de sospecha, el estudiante deberá justificar su código y demostrar su desarrollo individual.

- Penalizaciones

- Falta de Documentación: -100%
- Uso de Javascript para la comunicación entre backend/frontend: -20pts
- No utilizó Flask (Backend): -100%
- No utilizó Django (Frontend): -100%
- Falta de release: -10% por cada release faltante
- No utilizó POO: -100%

8.5 Comentarios Generales
